DOCUMENTACIÓN TÉCNICA DEL SISTEMA INTEGRAL: BOOKSY LITERARIO

Desarrollado por: Cristian Daniel Villalba Marín

Fecha: 6 de marzo de 2026

Versión: 1.0.0.

DOCUMENTACIÓN TÉCNICA DEL SISTEMA INTEGRAL: MEETUP LITERARIO..... 1

1. INTRODUCCIÓN AL PROYECTO.....3

1.1. Propósito del Documento..... 3

2. ARQUITECTURA DEL SISTEMA (TECH STACK)..... 4

2.1. Backend: Laravel 11 (PHP 8.3+)..... 4

2.2. Frontend Web: Angular (v17+)..... 4

2.3. Aplicación Móvil: Flutter (Dart)..... 4

3. MODELO DE DATOS Y PERSISTENCIA.....4

3.1. Estructura de la Entidad User..... 4

3.2. Relaciones del Modelo..... 5

4. LÓGICA DE NEGOCIO EN EL BACKEND.....5

4.1. Creación Blindada de Usuarios (store)..... 5

4.2. Serialización y Casts de Laravel 11.....5

5. SEGURIDAD Y CONTROL DE ACCESO.....6

5.1. El Búnker Administrativo (Angular)..... 6

5.2. Resolución del Conflicto de Hashing.....6

6. IMPLEMENTACIÓN DE LA APLICACIÓN MÓVIL (FLUTTER)..... 6

6.1. Sincronización de Datos (Pull-to-Refresh)..... 6

6.2. Adaptación de Formatos de Fecha..... 7

7. REGISTRO DE INCIDENCIAS Y SOLUCIONES TÉCNICAS..... 7

7.1. Incidencia: Error 401 Unauthorized en el Panel Admin.....7

7.2. Incidencia: Flutter Home no cargaba con nuevos Administradores..... 7

8. MANUAL DE OPERACIONES PARA EL ADMINISTRADOR.....7

8.1. Gestión de Usuarios.....8

8.2. Gestión de Eventos..... 8

1. INTRODUCCIÓN AL PROYECTO

La plataforma "Meetup Literario" es una solución tecnológica avanzada diseñada para la gestión, organización y participación en eventos culturales y literarios. El ecosistema permite la interacción entre administradores y usuarios finales a través de una arquitectura distribuida que garantiza la integridad de los datos, la seguridad de los accesos y una experiencia de usuario fluida en múltiples dispositivos.

1.1. Propósito del Documento

Este documento técnico, redactado por **Cristian Daniel Villalba**, tiene como objetivo detallar la arquitectura del sistema, los desafíos técnicos superados durante el desarrollo, la estructura de la base de datos y la lógica de negocio aplicada en el Backend, Frontend Web y Aplicación Móvil.

---

2. ARQUITECTURA DEL SISTEMA (TECH STACK)

El sistema se basa en una arquitectura de microservicios desacoplados para permitir un mantenimiento independiente de cada capa.

2.1. Backend: Laravel 11 (PHP 8.3+)

Se ha seleccionado Laravel 11 como el motor central del API. Esta versión introduce mejoras significativas en la eficiencia de los controladores y la gestión de modelos.

- **Eloquent ORM:** Utilizado para la abstracción de la base de datos.
- **Laravel Sanctum:** Implementado para la gestión de tokens Bearer, proporcionando una capa de seguridad ligera y eficiente para SPAs y aplicaciones móviles.
- **Hashing:** Uso nativo de algoritmos Bcrypt para la protección de credenciales.

2.2. Frontend Web: Angular (v17+)

La interfaz de administración se ha desarrollado bajo una arquitectura de **Standalone Components**, eliminando la redundancia de los módulos tradicionales de Angular.

- **Bootstrap 5:** Para el diseño visual y la adaptabilidad (Responsive Design).
- **RxJS:** Para el manejo de flujos de datos asíncronos y peticiones HTTP.

2.3. Aplicación Móvil: Flutter (Dart)

El cliente para usuarios finales está construido en Flutter, permitiendo un rendimiento nativo en Android e iOS desde una única base de código.

- **Gestión de Estado:** Patrón BLoC (Business Logic Component).
- **Parseo de Datos:** Implementación de la librería `intl` para el manejo de formatos de fecha regionales.

---

3. MODELO DE DATOS Y PERSISTENCIA

La base de datos centraliza toda la información del ecosistema. El modelo más complejo es el de `User`, el cual ha sido extendido para cubrir necesidades administrativas.

3.1. Estructura de la Entidad `User`

De acuerdo con la implementación de **Cristian Daniel Villalba Marin**, el modelo `User` en Laravel gestiona los siguientes campos críticos:

- `name` & `last_name`: Identificación completa del usuario.
- `email`: Identificador único para el acceso.
- `password`: Almacenado mediante el cast `hashed`.
- `role`: Clasificación de permisos (admin, user).
- `registration_date`: Fecha de alta personalizada con formato `d/m/Y H:i`.
- `status`: Estado operativo (active, inactive, suspended).
- `phone`, `reputation`, `category_preferences`, `access_level`, `responsibility_area`: Metadatos adicionales para perfilado de usuario.

3.2. Relaciones del Modelo

El sistema utiliza relaciones Eloquent avanzadas para vincular usuarios con el resto de entidades:

- **HasMany (Books):** Libros subidos por el usuario.
- **HasMany (Transactions):** Registro de compras y ventas.
- **BelongsToMany (Meetups):** Participación en eventos gestionada mediante una tabla pivote `meetup_user`.

---

4. LÓGICA DE NEGOCIO EN EL BACKEND

El controlador `UserController` actúa como el orquestador de las operaciones administrativas del panel web.

4.1. Creación Blindada de Usuarios (`store`)

El método `store` ha sido diseñado por **Cristian Daniel Villalba Marín** para garantizar la seguridad del panel administrativo:

1. **Validación Robusta:** Se utiliza el facade `Validator` para asegurar que cada campo cumple con los requisitos mínimos de tipo y longitud.
2. **Instanciación Manual:** Se evita el uso de `User::create` para saltar las restricciones de asignación masiva y asegurar que el campo `role` siempre se guarde como `admin` cuando se crea desde la web.
3. **Contraseña Predeterminada:** Implementación de una lógica de contingencia que asigna `password123` si el administrador no especifica una contraseña en el formulario.

4.2. Serialización y Casts de Laravel 11

Para facilitar el trabajo del frontend, se han implementado casts personalizados en el modelo `User`:

- **Fecha de Registro:** Se transforma automáticamente de ISO-8601 a un formato legible español (`d/m/Y H:i`).
- **Hasheado Automático:** El cast `hashed` asegura que cualquier asignación al campo password sea encriptada antes de persistir en la base de datos.

---

5. SEGURIDAD Y CONTROL DE ACCESO

Uno de los hitos más importantes logrados por **Cristian Daniel Villalba Marín** ha sido la protección de acceso basada en roles.

5.1. El Búnker Administrativo (Angular)

Para evitar que usuarios normales de la aplicación móvil accedan al panel web, se ha implementado un filtro de seguridad en el login:

1. **Login Request:** El frontend envía las credenciales a Laravel.
2. **Rol Validation:** Al recibir la respuesta exitosa, Angular verifica el campo `user.role`.
3. **Expulsión Automática:** Si el rol no es estrictamente `admin`, se invoca el método `logout()`, se limpia el `localStorage` y se muestra un mensaje de "Acceso Denegado".

5.2. Resolución del Conflicto de Hashing

Durante el desarrollo se detectó un error 401 provocado por un "Doble Hash". La solución aplicada consistió en eliminar la llamada manual a `Hash::make()` en el controlador,

confiando plenamente en la automatización del modelo `User`, garantizando así que las contraseñas guardadas coincidan con el login del usuario.

---

6. IMPLEMENTACIÓN DE LA APLICACIÓN MÓVIL (FLUTTER)

La aplicación móvil ofrece una experiencia dinámica centrada en la visualización de eventos.

6.1. Sincronización de Datos (Pull-to-Refresh)

Para asegurar que los usuarios vean los cambios realizados desde el panel de Angular en tiempo real, se implementó el widget `RefreshIndicator`.

- **Mecánica:** El gesto de tirar hacia abajo dispara una petición GET fresca a la API de Laravel.
- **Física de Scroll:** Se aplicó `AlwaysScrollableScrollPhysics` para permitir la recarga incluso cuando no hay elementos en la lista, eliminando la sensación de "app congelada".

6.2. Adaptación de Formatos de Fecha

Debido al cambio de formato en Laravel para favorecer al panel web, la aplicación de Flutter requirió un ajuste en el modelo de datos.

- Se integró la librería `intl` para parsear el formato `dd/MM/yyyy HH:mm`, evitando errores de ejecución (`FormatException`) al procesar los datos del backend.

---

7. REGISTRO DE INCIDENCIAS Y SOLUCIONES TÉCNICAS

En esta sección, **Cristian Daniel Villalba Marín** documenta los desafíos técnicos más complejos resueltos durante la fase de desarrollo.

7.1. Incidencia: Error 401 Unauthorized en el Panel Admin

- **Descripción:** Los usuarios creados desde Angular no podían loguearse a pesar de usar las credenciales correctas.
- **Causa Raíz:** Se estaba aplicando `Hash::make` en el controlador sobre un modelo que ya tenía el cast `hashed` activo.

- **Resolución:** Se simplificó la asignación de contraseña en el controlador enviando el texto plano y permitiendo que Laravel gestionara la encriptación de forma única.

7.2. Incidencia: Flutter Home no cargaba con nuevos Administradores

- **Descripción:** Al cambiar el formato de fecha en Laravel, la app móvil se cerraba o mostraba errores rojos.
- **Causa Raíz:** Incompatibilidad entre el estándar ISO de Dart y el formato visual español de la API.
- **Resolución:** Se actualizó el parser del modelo en Dart para soportar patrones de fecha específicos mediante `DateFormat`.

---

8. MANUAL DE OPERACIONES PARA EL ADMINISTRADOR

Instrucciones para la gestión eficiente de la plataforma por **Cristian Daniel Villalba Marín**.

8.1. Gestión de Usuarios

1. Acceder a la sección "Usuarios" del panel.
2. Para crear un administrador, pulsar "Añadir Usuario".
3. Completar obligatoriamente nombre, apellido y correo electrónico.
4. Si se desea asignar una contraseña específica, escribirla en el campo correspondiente; de lo contrario, informar al usuario que su contraseña será `password123`.

8.2. Gestión de Eventos

- **Edición:** Al pulsar "Editar", el sistema cargará un formulario reactivo con los datos actuales del evento. Al guardar, se realizará una petición PUT a la base de datos.
- **Recarga:** Los cambios guardados se reflejarán instantáneamente en la app móvil una vez que el usuario realice el gesto de pull-to-refresh.